



How to Use Git: A Beginner's Guide

A learning companion for the Real Python "How to Use Git" tutorial. Visit realpython.com to learn more.

Getting Started

1. Install & Verify Git

Check if Git is already installed:

```
$ git --version
git version 2.24.0
```

If not installed, download from git-scm.com/install or use a GUI client (GitHub Desktop, Sourcetree, GitKraken).

Git vs GitHub

They're Not the Same!

Git	GitHub
Version control software	Online platform
Runs locally on your computer	Hosts Git repos in the cloud
Tracks file changes & history	Adds collaboration & sharing
Works offline, no account needed	Requires internet + free account
Key Point: You don't need GitHub to use Git!	

Create & Initialize Your Project

2. Create a Project Directory

```
$ mkdir hello-git/
$ cd hello-git/
```

Add a Python file to track:

```
# main.py
def greet(name):
    return f"Hello, {name}!"
print(greet("World"))
```

3. Initialize a Git Repository

```
$ git init
```

Creates a hidden `.git/` folder that stores your project's history and settings.

```
$ git status
On branch main
Untracked files:
  main.py
```

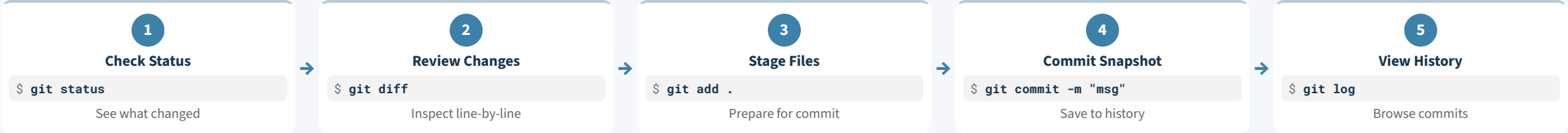
4. Add a `.gitignore`

Tell Git which files to skip:

```
__pycache__/
*.pyc
.env
venv/
```

Tip: GitHub maintains a ready-made Python `.gitignore` template.

The Core Git Workflow



Staging & Committing

Stage Files

The staging area is a checklist of changes for your next snapshot.

```
# Stage one file
$ git add main.py

# Stage everything
$ git add .

# Check what's staged
$ git diff --staged
```

Caution: Run `git status` before `git add .` to avoid staging unwanted files.

Reading Git Diff Output

Understanding Diffs

`git diff` shows line-by-line changes vs. the last commit:

```
@@ -1,4 +1,4 @@
def greet(name):
    return f"Hello, {name}!"

-print(greet("World"))
+print(greet("Git learner"))

-Red = removed +Green = added
Note: git diff only shows tracked files.
```

Anatomy of a Commit

What's Inside a Commit?

- Commit Hash** -- Unique ID (`1a1da40`)
- Author** -- Name and email
- Date & Time** -- When created
- Message** -- What changed
- Snapshot** -- Full state of tracked files

```
$ git log
commit 1a1da40 (HEAD -> main)
Author: You <you@email.com>
Initial commit
```

Writing Good Commit Messages

Commit Message Best Practices

- Keep first line under **50 characters**
- Use **imperative mood**: "Add feature" not "Added"
- Be specific: "Add user auth" not "Update code"

```
# Short message
$ git commit -m "Add greeting"

# Multi-line (opens editor)
$ git commit
```

Tip: If Vim opens: `i` to type, `Esc` then `:wq` to save.

Managing Files

Rename & Recover

Rename files so Git tracks them properly:

```
$ git mv main.py greet.py
$ git commit -m "Rename file"
```

Undo mistakes:

```
# Unstage a file
$ git restore --staged <file>

# Discard local changes
$ git restore <file>
```

Warning: `git restore` discards changes permanently!

Quick Command Reference

Command	What It Does	Command	What It Does
<code>git init</code>	Turn a folder into a Git repo	<code>git diff</code>	Show unstaged changes line by line
<code>git status</code>	Show tracked, modified, staged files	<code>git diff --staged</code>	Show staged changes ready to commit
<code>git add <file></code>	Stage a file for the next commit	<code>git log</code>	View commit history
<code>git add .</code>	Stage all changes in the directory	<code>git mv <old> <new></code>	Rename a file and stage the change
<code>git commit -m "msg"</code>	Save a snapshot with a message	<code>git restore <file></code>	Discard uncommitted changes
<code>git branch</code>	List branches, show active one (*)	<code>git restore --staged <f></code>	Unstage without losing changes